
Detecting Simplification in Neural Network Dynamics from Scalar Logs: Supplementary Materials

Anonymous Author

Anonymous Institution

A THEORY, IMPLEMENTATION, AND EXPERIMENTAL DETAILS

A.1 Dimension preservation under delay reconstruction

The dimension interpretation has two steps. First, delay-embedding theorems give conditions under which a generic scalar observable can reconstruct the state of a deterministic system. Second, a local dimension estimator approximates the geometry of that reconstruction from finite data. The reconstruction theorem alone does not guarantee accurate estimation; the estimator alone does not establish an embedding.

A.2 Recurrent, transient, and stochastic regimes

Figure 1 makes the assumptions operational. A recurrent trajectory returns to nearby states, a transient moves through a region without repeated visits, and stochastic forcing fills the observed space according to the noise and the observation channel. The same numerical estimator returns a value in all three cases; the regime determines whether that value can be called a dimension.

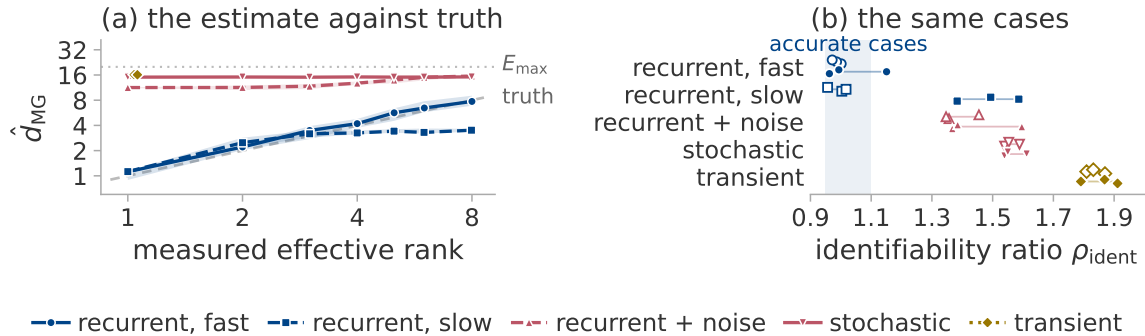


Figure 1: Regime diagnostics from the synthetic study. The left panel compares MG with an independently measured effective rank; the right panel separates recurrent and non-recurrent records using identifiability. The shaded region marks the fixed regime in which the component interpretation is supported.

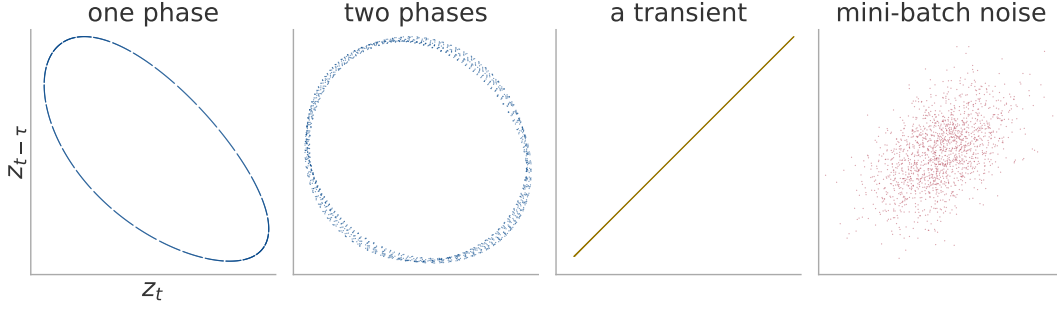


Figure 2: Delay reconstructions of four cases. A recurrent one-phase signal closes a loop; two phases fill a higher-dimensional set; a transient follows a path without returning; stochastic forcing fills the available region. These pictures explain why recurrence and identifiability are required before reading MG as active dimension.

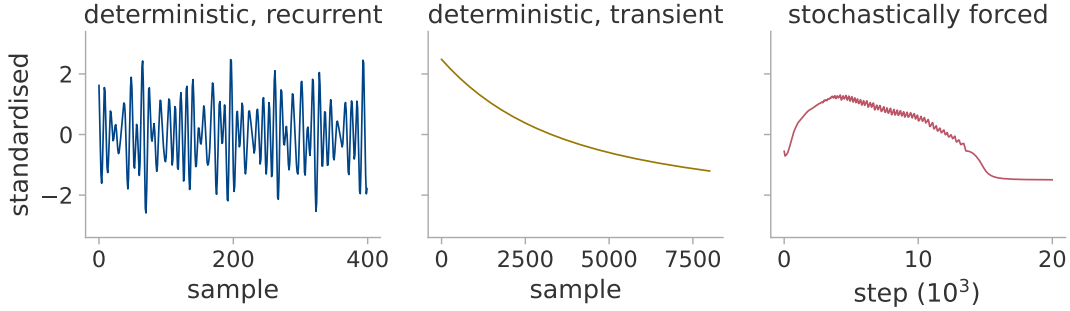


Figure 3: The same scalar-log viewpoint across regimes: recurrent forcing, a transient optimization path, and a stochastically forced training record. The estimator is applied to all three, while the interpretation changes with the regime.

A.3 Pooled estimator: implementation steps

For reproducibility, the implementation can be summarized as follows. The dither and numerical floors only handle ties and degeneracy; they do not provide a statistical correction. The implementation standardizes first, adds Gaussian dither with scale 10^{-9} , floors distances at 10^{-8} and local sums at 10^{-5} , and marks a window when more than 1% of either quantity reaches a floor.

Input: scalar record $x_{1:N}$, lag τ , embedding dimension E , number of neighbors k , Theiler exclusion T .

1. Reject nonfinite, constant, or too-short records. Standardize x_t and add fixed-scale dither, giving z_t .
2. Form $y_t = (z_t, z_{t+\tau}, \dots, z_{t+(E-1)\tau})$.
3. For each y_i , find the k closest y_j with $|i - j| > T$. Return an unavailable estimate if neighbors are insufficient.
4. Accumulate $S_i = \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})$, recording distance and sum floors.
5. Set $n = N - (E - 1)\tau$, the number of valid delay vectors. Return $\hat{d}_{\text{MG}} = (n(k - 1) - 1) / \sum_{i=1}^n S_i$ and the degeneracy flags.

The change detector is separate from the estimator: it compares recent and earlier blocks of window estimates. This separation lets us compare MG with other scalar features under the same causal decision rule.

Proposition 1 (Bi-Lipschitz invariance). *Let μ have local dimension d at s , and let F be bi-Lipschitz on its support, with constants $0 < c \leq C < \infty$. Then the pushforward $F_{\#}\mu$ has local dimension d at $F(s)$.*

Proof. For sufficiently small ϵ , the inverse image of $B(F(s), \epsilon)$ is bounded on the support between $B(s, \epsilon/C)$ and $B(s, \epsilon/c)$. Their probabilities bound the pushforward probability. The logarithms of c and C contribute vanishing constants after division by $\log \epsilon$, so both bounds have the same limiting exponent d . $\square$

Table 1: Primary configurations. An exclusion of “span” is $(E - 1)\tau$. The adaptive generator lag is one quarter of the autocorrelation time, capped so the delay span occupies at most one eighth of the window; its exclusion is capped at 320. Alternative settings are reported as sensitivity checks, not substituted for the primary results.

Experiment	Observable	W	E	τ	k	Theiler exclusion	Stride
Digits head	state-based	8,000	20	4	20	span; cap 150	3,000
Regression	loss / probe	2,048; 8,192	16	1	10	100	endpoints
Generator	neuron zero	8,192	20	adaptive	20	ACF; cap 320	8,192
CNN / ResNet-18	parameter norm	1,000	20	1	20	span	500
VAE	probe NLL	512	20	1	20	39	128
Walker2d	joint / action scalar	2,048	20	8	20	312	512 / 1,024

This elementary property states the geometric assumption behind our interpretation. A smooth embedding of a compact smooth manifold provides the needed local regularity. For a fractal support, an injective map alone need not provide the required reference of distances. The forced-system results of Stark (1999); Stark et al. (2003) impose additional conditions and include fibre-wise formulations. Their application to a stochastic optimizer would require checking those conditions; it does not follow from observing a scalar training log.

A.4 Affine invariance and its implementation boundary

Proposition 2 (Affine invariance of the ideal scalar estimate). *For a finite nonconstant record, fixed delay and neighbor settings, and no numerical dither, the MG formula is unchanged by replacing every observation with $ax_t + b$, for any $a \neq 0$, provided the estimate is defined.*

Proof. After centering and dividing by standard deviation, the transformed series is the original standardized series multiplied by $\text{sign}(a)$. All pairwise Euclidean distances of its delay vectors are identical. Temporal exclusions and admissible neighbors therefore remain unchanged, as do all distance ratios and their pooled estimate. $\square$

The implemented code adds Gaussian dither of scale 10^{-9} after standardization, floors distances at 10^{-8} and local log-ratio sums at 10^{-5} , and records the fractions affected by these floors. Positive affine changes preserve the standardized input in exact arithmetic and preserve a run with the same dither. A negative rescaling preserves the distribution under symmetric dither, but need not give exactly equal values with the same realized dither. Finite precision and almost-constant records require separate checks. The proposition concerns a whole-window affine transformation: a scale change within a window is a different time series.

A.5 Pooling and dependence

The numerator $n(k - 1) - 1$ is the bias correction used by the existing implementation. Its Gamma-model motivation assumes idealized independent local statistics. Neighbor reuse and serial dependence mean that this correction is not a general finite-sample unbiasedness guarantee for the records in this paper. We use the same implementation throughout, preserve degeneracy flags, and do not clip large estimates to E . The Theiler exclusion removes temporally adjacent candidates but does not make the remaining observations independent.

B CONFIGURATIONS AND REPRODUCIBILITY

Tables and figures are generated from saved CSV/JSON experimental outputs. The accompanying source bundle records file hashes and maps numerical claims to their sources. We specify the observable, estimator settings, and development/test split for each task. All tasks use the same estimator; window length and lag are chosen for the relevant timescale.

Exact adaptive rules. For centered samples z_t , define $a(\ell) = \sum_{t=1}^{W-\ell} z_t z_{t+\ell} / \sum_{t=1}^W z_t^2$. The autocorrelation time t_a is the first lag with $a(\ell) < e^{-1}$, or W if none exists. The adaptive generator uses

$$\begin{aligned}\tau_0 &= \max(1, \text{round}(t_a/4)), \\ \tau &= \min\{\tau_0, \max(1, \lfloor W/[8(E - 1)] \rfloor)\}.\end{aligned}$$

The implementation uses nearest-even rounding. Theiler settings resolve to $T = \min(T_{\max}, T_0)$, where $T_0 = (E-1)\tau$ for the embedding rule and $T_0 = \max((E-1)\tau, t_a)$ for the autocorrelation rule. An explicit integer is also capped. Thus a cap can shorten the requested exclusion below the embedding span; report both the rule and its resolved value. The primary digits configuration uses the embedding rule (76 samples), not an uncapped autocorrelation rule.

The historical digits sweep uses stride 3,000 on the 26,000 post-burn samples ($\max(500, \lfloor (N-W)/6 \rfloor)$); development used stride 2,000. The known-mode table uses endpoint windows; its additional switch trace uses $W = 4,096$ and stride 2,048. Walker uses stride 512 in the early repair series and 1,024 in later three-window summaries. Generator windows are disjoint. The cyclic VAE uses stride 64. These variations are fixed for each procedure, not selected after comparing outcomes.

Window length and recurrence requirements. A window must leave enough admissible neighbors after delay reconstruction and temporal exclusion. Meeting the software minimum is necessary but does not guarantee an accurate estimate. For dimension estimation, the window should contain repeated visits within one regime. The lag must also expose the phase structure: increasing record length cannot compensate for delay vectors that cover only a nearly linear segment. For detection, longer windows stabilize estimates but blur transitions. We report detection delay to make this tradeoff explicit.

Units of replication. The CNN interventions share seeds and pre-intervention histories; rescaling and smoothing are transformations of unchanged logs. Generator tasks reuse initialization seeds. The VAE branches share initial trajectories, data, and probes. The Walker continuations share a learned starting policy. We report detection totals and paired contrasts at these levels; we do not treat overlapping windows or transformed logs as independent draws from a population. The VAE intervals are percentile bootstrap intervals over nine paired confirmation seeds, using 20,000 resamples and bootstrap seed 20260929. They measure initialization/batch variation conditional on the selected data and observable.

C DEVELOPMENT SYSTEMS AND MLP-BASED RECOVERY

C.1 Analytic and driven-learning evaluation

The synthetic suite includes an analytic matrix with independent oscillatory entries and learning systems with time-dependent targets or forcing: online regression, logistic regression, a frozen nonlinear decoder, and an MLP in a parameter subspace. To distinguish input variation from learned motion, we set the learning rate to zero and check whether each observable still changes. Several early loss and gradient observables continue varying. Their dimension estimates can recover the forcing structure, but cannot be attributed solely to changing parameters. The digits-head experiment below addresses this ambiguity with state-based observables that become constant when learning stops.

Table 2: Controlled validation with each experiment’s reference. Mean absolute error (MAE) averages rank–seed–observable combinations. The driven-target MLP signal still varies with frozen parameters; its error describes the driven signal, not parameter motion alone.

Construction	Range	MAE
Analytic oscillating matrix	1–8	0.27
Driven-target MLP subspace	1–20	0.92
Digits head, covariance reference	1–8	0.90

In the digits experiment, we train and freeze an MLP feature extractor, then train a linear head in a fixed affine parameter subspace. The cross-entropy loss assigns weights to twelve fixed data groups. For the historical digits experiment, weights are $w_i(t) = \max\{0.05, 1 + 0.8 \sum_{j=1}^r G_{ji} \sum_{\ell=1}^r A_{j\ell} \sin(2\pi f_\ell t + \phi_\ell)\}$. Here G identifies the twelve fixed data groups and A equalizes their linearized forcing directions and gains. For $r > 1$, $f_j = 2^{a_j}/16$, with $a_j = 0.03 + 0.94(b_j - \min b)/(\max b - \min b)$ and $b_j = \text{frac}(\sqrt{p_j})$ for the first r primes; for $r = 1$, $f_1 = \sqrt{2}/16$. Phases are uniform on $[0, 2\pi)$ from NumPy’s generator seeded by $31 + \text{seed}$; group assignment uses $7717 + \text{seed}$. The drive amplitude is 0.8 and the preconditioned head step is 0.15. These describe the archived implementation behind the reported scores; the newer code organizes random streams differently. Development uses seeds 90–92

and ranks 2,4,6; evaluation uses seeds 0–3 and held-out ranks 1,3,5,8. The selected settings are $E = 20$, $\tau = 4$, $k = 20$, and an 8,000-point window. This configuration lies on the boundary of the search grid and is held fixed for evaluation; the search does not establish a global optimum.

All usable state-based observables order the held-out ranks correctly. The mean error against measured covariance effective rank is 0.90 across individual records; aggregating by rank before computing the error gives 0.382. We report the former because these aggregations answer different questions. Errors by observable are 0.60 for fixed-probe loss, 0.41 for parameter norm, 0.34 for a fixed parameter projection, and 1.81 for gradient norm. Instantaneous mini-batch loss continues varying with frozen parameters, while quantized probe accuracy is degenerate; neither supports the parameter-dimension interpretation. The specified phases define the intended structure, and the covariance reference checks which directions are measurably excited.

C.2 Dense regression with endogenous modes

The data matrix has 256 rows and 32 columns, full column rank, and Hessian $H = X^\top X/256$. Orthogonal rotations make the data and learned weights dense. Eigenvalues are grouped into r repeated values, with a single excited direction within each group. For heavy-ball momentum one and step size one, the error in group j obeys

$$q_{j,t+1} = 2 \cos(\omega_j) q_{j,t} - q_{j,t-1}, \quad \lambda_j = 2(1 - \cos \omega_j).$$

The selected irrational frequencies produce r independent phases. All 32 original coordinates move. Three seeds vary bases, teacher directions, and probes, but the normalized training-loss dynamics are identical up to rounding for a fixed spectrum. These are coordinate/observer checks rather than three independent loss processes.

Table 3: Known-mode regression: median scalar MG. Settings are fixed across r . A longer record does not uniformly reduce bias.

r	Loss, 2,048	Loss, 8,192	Probe, 8,192
1	0.90	1.12	1.01
2	2.39	2.57	2.55
4	3.71	4.99	4.11
6	5.88	7.00	4.98

Selective damping sets momentum to 0.98 in two of four groups at step 8,192. The comparison uses disjoint 8,192-point windows before damping and after the transition, with the latter starting at 12,288. Across three seeds, loss-based MG changes from 4.97 to 2.56 in every run; the median probe estimate changes from 4.15 to 2.63. The corresponding loss values are deterministic up to the tested seed changes in coordinates and probes, so the table reports the three-run range rather than treating the repeated loss curves as independent trajectories. With ordinary momentum 0.99 on all groups, learning converges and late losses reach rounding scale. Standardization can then produce spurious high estimates; the absolute pre-standardization scale must be checked.

Table 4: Three-seed variation in the selective-damping regression experiment.

Signal	Phase	Median	Minimum	Maximum
loss	before	4.971432	4.971432	4.971432
loss	after	2.562791	2.562791	2.562791
probe error	before	4.153513	3.601192	4.206165
probe error	after	2.634185	1.995991	2.680959

C.3 Learned autonomous generators

The 1,000-unit generator evolves as

$$\begin{aligned} \mathbf{h}_{t+1} &= 0.9\mathbf{h}_t + 0.1(J \tanh \mathbf{h}_t + U \mathbf{z}_t), \\ \mathbf{z}_t &= A^\top \tanh \mathbf{h}_t. \end{aligned}$$

Here $\mathbf{h}_t$ is the network state, J the recurrent matrix with gain 1.2, U the output-feedback matrix, and A the readout trained by FORCE/RLS for 100,000 simulation steps. Frequencies are proportional to $1, \sqrt{2}, \sqrt{3}, \sqrt{5}$ for

T1–T4; H2/H4 use integer harmonics, and M4 uses $1, 2, \sqrt{2}, 2\sqrt{2}$. All seven tasks use seeds 1–5. After training, we discard 5,000 burn-in steps and record 32,768 autonomous samples. The reported MG value is the median over four disjoint 8,192-point windows on neuron zero. Neurons one and two provide fixed robustness checks.

The Lyapunov reference uses twelve tangent vectors and a near-zero threshold of -0.02 chosen in advance. The number of near-zero exponents agrees with the target phase dimension in 31/35 cases. H2 has weakly contracting directions near the threshold and is overestimated in four seeds. Thus even this reference has finite-time ambiguity. MG has finite-resolution error as well: one periodic H2 record gives 0.73, T2 has median 2.84, and T3/T4 overlap. The reliable comparison is the distinction between phase structures, especially in the harmonic references; exact recovery across $d = 1-4$ is not established.

As a robustness check, we re-evaluated the stored H4, M4, and T4 generators on three neurons, two windows, and three additive-noise levels. On neuron zero, MG preserved the expected $H4 < M4 < T4$ ordering on all five seeds for 4,096 samples with no added noise and with noise of standard deviation 0.01 times the signal standard deviation. At noise level 0.05, the ordering held on two of five seeds for the shorter window and on none for the longer window. The corresponding median estimates were (1.14, 4.56, 6.30) in the clean short-window condition and (5.53, 5.12, 6.75) in the noisy short-window condition. This stress test supports robustness to small observation noise, while making the breakdown at larger noise explicit.

For the separate 256-unit motion experiment, gain is 1.5 and FORCE learns a figure-eight target for 40,000 simulation steps. Each frozen checkpoint is evaluated after 2,000 burn-in steps over 8,192 recorded states. The chaotic-start cohort consists of the first five eligible seeds among sequential candidates 6–30, selected by a positive pretraining Lyapunov criterion; thirteen candidates were examined, yielding seeds 7,9,15,16,18. No MG value enters selection. The unselected initial seeds 1–5 are retained separately because some start nonchaotic. This is conditional evidence for learning from chaos, not an estimate of how often random networks become chaotic or learn a stable cycle.

D APPLIED TRAINING PROCEDURES AND FULL COMPARISONS

D.1 CNN and ResNet interventions

The small CNN has three convolutional layers with 16, 32, and 32 ReLU channels, 3×3 kernels, max pooling after the first two layers, adaptive average pooling, and a ten-class linear head; it has 14,666 parameters and no normalization. It uses 10,000 CIFAR-10 images, SGD step 0.02, momentum 0.9, weight decay $5 \cdot 10^{-4}$, and batches of 64 sampled with replacement. The graded study runs 10,000 updates with an intervention at update 4,000. The before/after summaries use windows lying within the designated pre/post intervals, excluding the immediate mixed transition.

The learning-rate interventions divide the step size by 3, 10, or 100. Freezing leaves the later layers, only the head, or only its bias trainable. Pruning masks 50%, 80%, or 95% of weights. Some interventions rebuild SGD for the remaining trainable parameters, which also changes optimizer state. Their measured effect therefore includes that change and cannot be attributed to the number of parameters alone. Increased-batch references multiply batch size by four. Rescaling multiplies the base log by ten after update 4,000. Smoothing replaces that suffix by the causal 16-sample moving average $\bar{x}_t = \sum_{j=0}^{15} x_{t-j}/16$, using the preceding original samples at the boundary; neither transformation retraining the network.

The CNN IAAFT comparison uses two surrogates per window and reports their median MG, with RNG seed equal to the window start. Each surrogate uses 100 alternating spectrum/rank projections. Endpoint matching can trim up to 15% of the window before randomization, so spectrum and marginal preservation refer to the retained segment; it is a descriptive reference, not a matched-length significance test.

Detector development searches $M \in \{2, 3, 4\}$ and $B \in \{3, 4, 6\}$. For each pair, its drop threshold is the smallest giving no false positives on development nulls, plus a margin of 0.02. The selected pair maximizes development event detection, with shorter delay as a tie breaker. Companion-statistic directions are chosen on development data as well. The rule is frozen for the new graded seeds and for ResNet. The selected MG threshold is 0.0882988401. The primary 27/28 result covers seven strong intervention types on four seeds; weak types are reported separately. The 16 references comprise base, increased batch, rescaled base, and smoothed base on the same four seeds. Thus these totals describe a matched benchmark rather than independent Bernoulli trials.

Table 5: CNN interventions at all tested strengths, on four held-out seeds. Changes are medians of paired record ratios. Update covariance and the fraction of moving parameters provide independent comparisons. Stronger pruning does not always produce a larger MG decline.

Intervention	MG before	MG after	MG change	Update PR change	Moving parameters
No intervention	4.31	4.32	+0.4%	+40.3%	100.00%
Batch $\times 4$	4.31	4.85	+12.8%	-7.3%	100.00%
Log $\times 10$	4.31	4.32	+0.4%	+40.3%	100.00%
Smoothed log	4.31	4.44	+3.3%	+40.3%	100.00%
Step / 3	4.31	4.35	+0.9%	-5.4%	100.00%
Step / 10	4.31	3.95	-8.3%	-21.6%	100.00%
Step / 100	4.31	3.85	-11.5%	-20.9%	100.00%
Freeze first two convolutions	4.31	4.27	-1.8%	-13.6%	65.31%
Train head only	4.31	3.51	-18.5%	-43.5%	2.25%
Train head bias only	4.31	3.58	-16.7%	-68.4%	0.07%
Prune 50%	4.31	3.80	-12.7%	+14.7%	50.30%
Prune 80%	4.31	3.59	-17.5%	-10.3%	20.48%
Prune 95%	4.31	3.79	-11.5%	-45.3%	5.55%

D.2 Natural scalar baselines

We computed additional scalar features on the same saved parameter-norm records. Permutation entropy measures the diversity of local ordinal patterns (Bandt and Pompe, 2002); sample entropy measures the recurrence of similar short templates (Richman and Moorman, 2000); normalized increments measure local motion; and spectral entropy summarizes the detrended periodogram. We also tested a one-sided Page CUSUM (Page, 1954) on each window feature. Every feature used the same event definition, development seeds, detector family, and held-out seeds as MG. These comparisons test complementarity, not population-level superiority.

Table 6: Natural scalar features on the held-out CNN benchmark. Each detector is fixed on the earlier seeds. Successful detections cover seven strong intervention types across four seeds; false positives cover four reference types across four seeds.

Feature	Hits	False positives	Median delay
MG	27/28	0/16	1,000
Abs. increments	27/28	0/16	500
Norm. increments	28/28	0/16	1,500
Perm. entropy	28/28	7/16	1,500
Sample entropy	28/28	4/16	1,500
Spectral entropy	4/28	2/16	2,750
Trend crossings	15/28	5/16	1,000
Lag-one corr.	0/28	0/16	—
Detrended std.	28/28	10/16	500
Level	4/28	0/16	1,000

Table 7: False positives by type for the block detector. The reference records share the four held-out seeds; the table therefore describes this matched benchmark rather than independent false-positive probabilities.

Feature	Base	Batch $\times 4$	Rescaled	Smoothed
MG	0	0	0	0
Abs. increments	0	0	0	0
Norm. increments	0	0	0	0
Perm. entropy	3	0	3	1
Sample entropy	1	0	1	2
Spectral entropy	0	2	0	0
Trend crossings	1	2	1	1
Lag-one corr.	0	0	0	0
Detrended std.	2	2	4	2
Level	0	0	0	0

The primary block detector uses $W = 1,000$, stride 500, and excludes feature windows ending at or before update 1,500 from the detection history. MG uses $M = 2$, $B = 3$, and a drop threshold of 0.08829884; each baseline selects its direction and threshold on the development records. Permutation entropy uses ordinal order five, unit delay, stable sorting for ties, and normalization by $\log(5!)$; sample entropy uses template length two,

Table 8: Selected detector parameters. The block sign is listed; all selected CUSUM signs are recorded in `new_results/rules.json`. Full-precision values in that file define the implemented thresholds.

Feature	M	B	Sign	δ	CUSUM drift	CUSUM h
MG	2	3	1	0.0882988	0.25	44.3109
Abs. increments	2	3	1	0.338305	1.0	19.7929
Norm. increments	4	4	-1	0.0289438	0.25	5.90329
Perm. entropy	2	6	-1	0.339038	1.0	7.79449
Sample entropy	2	3	-1	0.186699	0.25	9.86063
Spectral entropy	3	3	-1	1.29961	0.25	6.74364
Trend crossings	2	4	1	0.662857	0.5	2.31626
Lag-one corr.	2	3	1	0.0238814	0.25	0.653369
Detrended std.	2	3	-1	0.305439	0.25	0.5
Level	2	3	1	0.02	0.25	0.5

unit delay, Chebyshev distance, tolerance 0.2 times the window standard deviation, and self-match exclusion; normalized increments divide mean absolute increments by the window standard deviation; spectral entropy uses the normalized detrended periodogram; trend crossings count sign changes after linear detrending. Page CUSUM uses the same features, a past-only warm-up, drift selected from $\{0.25, 0.5, 1\}$, and the parameters in Table 8. The block score is $s(1 - \text{median}_M / \text{median}_B)$, with direction $s \in \{-1, 1\}$; MG fixes $s = 1$. Development searches $M \in \{2, 3, 4\}$, $B \in \{3, 4, 6\}$. Its threshold exceeds the largest reference score by 0.02, and selection maximizes event detections, then minimizes median delay. A baseline ratio is skipped when the reference magnitude is at most 10^{-12} .

CUSUM fixes its mean μ_0 and scale $\sigma_0 = \max(\text{std}_0, 0.01|\mu_0|, 10^{-10})$ from feature windows ending at 2,000–3,500. Starting at zero, $C_j = \max\{0, C_{j-1} + s(\mu_0 - v_j)/\sigma_0 - \kappa\}$; a detection occurs when $C_j > h$. Both directions and the stated drift grid are fixed, with h exceeding the largest development-reference score by 0.5. These rules are causal but warm-up must finish before scoring; they are not detectors available from the first training step.

Table 9: Past-only CUSUM on the held-out CNN records. Thresholds are selected on development records; the table reports strong-event detections, reference false positives, and median delay.

Feature	Detected	FP	Delay
MG	17/28	0/16	3,500
Abs. increments	27/28	12/16	2,500
Norm. increments	28/28	4/16	1,000
Perm. entropy	28/28	4/16	1,000
Sample entropy	28/28	4/16	1,250
Spectral entropy	5/28	1/16	500
Trend crossings	13/28	3/16	1,000
Lag-one corr.	1/28	0/16	500
Detrended std.	28/28	4/16	500
Level	0/28	0/16	—

On this benchmark, normalized increments detect 28/28 events with 0/16 reference false positives, compared with 27/28 and 0/16 for MG. Absolute increments match MG in detections and false positives with shorter median delay. Both entropy features detect all events but produce reference false positives. Simple features achieve higher detection rates in this CNN benchmark. MG provides a geometric statistic that is also evaluated on the specified generator, VAE, RL, and full-state timing benchmarks.

We evaluated ten additional seeds using a fixed procedure and historical detector rules fixed before training. This transfer test produced 19/30 intervention detections for MG; five of ten ordinary-training references contained threshold crossings, and six intervention branches had crossings before the assigned intervention time. Mean absolute increments produced 30/30 detections with no reference false positives. Normalized increments produced 29/30 detections and false positives in two of ten references. This confirmation indicates that the original CNN benchmark supports the fixed comparison above, but its detector threshold does not transfer universally across initialization and event schedules.

The MG horizon sensitivity on the original held-out records (Table 10) is 8/28, 16/28, 22/28, and 27/28 detections at horizons 500, 1,000, 2,000, and 5,000 updates, respectively; reference false positives remain 0/16 at all four horizons.

Table 10: Post-hoc horizon sensitivity with thresholds and first detections unchanged. Entries are detections within the specified number of updates; FP uses each complete reference record and is independent of the event horizon.

Feature	500	1,000	2,000	5,000	FP
MG	8/28	16/28	22/28	27/28	0/16
Abs. increments	15/28	16/28	26/28	27/28	0/16
Norm. increments	0/28	0/28	21/28	28/28	0/16
Perm. entropy	0/28	0/28	23/28	28/28	7/16
Sample entropy	8/28	13/28	27/28	28/28	4/16
Spectral entropy	0/28	0/28	0/28	4/28	2/16
Trend crossings	0/28	15/28	15/28	15/28	5/16
Lag-one corr.	0/28	0/28	0/28	0/28	0/16
Detrended std.	25/28	27/28	27/28	28/28	10/16
Level	0/28	4/28	4/28	4/28	0/16

ResNet-18 runs use the full CIFAR-10 training set, new seeds 20–22, and scratch/pretrained settings. SGD uses momentum 0.9, weight decay $5 \cdot 10^{-4}$, initial step 0.02, and batch 128 without augmentation; the pretrained setting uses 64×64 images and ImageNet normalization. The pretrained network retains the standard 7×7 , stride-two stem and max pool and replaces its classifier by a ten-class head. The scratch network uses a 3×3 , stride-one stem, no initial max pool, and 32×32 CIFAR-normalized inputs. MG and the rule transfer unchanged. A 1,024-coordinate CountSketch of updates provides a tractable covariance reference. The head-only intervention succeeds in both settings; learning-rate and pruning detection does not. The six freezing records and 24 references reuse three seeds and two initial-model settings.

D.3 ReLU-death stress test

A separate experiment uses six new seeds 30–35 and abrupt learning-rate shocks, with doses chosen on a pilot using activation death and accuracy, not MG. The tested learning-rate multipliers are 20, 30, 50, and 100 for one update; 15 for three updates; and 20 for two updates, followed by restoration of the base rate. A channel is classified as dead if it is zero on all 500 diagnostic images. An event requires an increase of at least 0.05 in the fraction of dead channels. There are 42 training runs, augmented by rescaled and smoothed base logs. Twenty-three runs meet the death criterion; two catastrophic whole-network failures are excluded from the prescribed eligible-event score and remain reported separately. The 42 runs comprise seven arms on six seeds: 23 meet the event criterion and 19 do not. Removing two catastrophic events leaves 21 eligible events; six rescaled and six smoothed base logs augment the 19 non-events to 31. Thus $42 - 2 + 12 = 52$ scored records, not 52 independent training runs.

Table 11: Boundary case: the transferred detector on ReLU death. Rules are fixed before the confirmation results.

Statistic	Detected / 21	False positives / 31
MG	12	12
Trend crossings	19	7
Loss stagnation	21	8
Detrended std.	21	25
Lag-one correlation	0	0

MG changes are nonmonotone in channel death: moderate loss of channels and catastrophic disruption can move the estimate in opposite directions. The CNN success therefore concerns the tested sustained training interventions and is not a generic failure detector.

D.4 Text VAE and preservation reference

We select 6,000 training and 256 fixed evaluation sentences with seed 20260929. Each has 8–24 words before the end-of-sentence token. A 2,000-token vocabulary is built from the training data. Shared embeddings have dimension 48, both GRUs have hidden size 64, and the diagonal Gaussian latent code has dimension 16. The code enters the decoder’s initial state and every decoding step. Training uses teacher forcing, Adam with learning rate

0.001, batch size 32, gradient-norm clipping at five, and no dropout. The loss sums reconstruction NLL within each sentence, adds β times the KL term, and averages over sentences. Since the encoder observes the sentence, this reconstruction score differs from marginal generative likelihood and ordinary language-model perplexity.

The branches share weights, Adam state, and batch/noise generators at update 1,024. Before every optimizer update, we evaluate 16 fixed evaluation sentences with fixed Gaussian noise (seed 829). Early MG is the median of five windows ending at updates 512–1,024; late MG is the median of nine windows ending at 2,048–3,072. Both use stride 128.

Independent references are measured every 128 steps. The empirical-mixture mutual information (MI) estimate uses four fixed draws from each of 256 posteriors and is bounded above by $\log 256$ nats. The code-response score is the symmetric KL divergence between decoder token distributions with matched and cyclically shuffled posteriors, using matched noise and teacher forcing. We record information loss when both references fall below half their reference values, provided their early values were nontrivial.

The preservation branch uses free bits: at $\beta = 1$, it floors the batch-averaged KL of each latent coordinate at 0.5 nats before summing. We select this intervention on pilot seed zero using only the independent references. An earlier pilot with five extra encoder updates fails to preserve information and is retained as a negative result. Preservation requires both late references to retain at least half their early values and exceed the suppressed branch by at least a factor of two. All nine test seeds pass. This is a third continuation of each shared initialization, so the comparisons remain paired.

Table 12: Text VAE: late medians across nine confirmation seeds. MI and code response are independent of the MG calculation.

Branch	MI	Code response	MG
Low regularization	5.545	2.087	8.129
Suppressed code	0.327	0.021	4.172
Free bits	3.934	0.599	5.491

The suppression result is consistent across all nine seeds at lag one for window lengths 256, 512, and 1,024. At lag four, seven of nine seeds agree. The 1,024-point configuration has only one early window. Using ordinary mini-batch loss instead of the fixed probe gives the expected direction on only four seeds. Standard deviation and spectral entropy of the fixed probe decline on all nine. These comparisons show that a stable probe is important for detecting the information change in this experiment.

D.5 Cyclic-VAE monitoring

Fresh seeds 100–109 use the same architecture/data. Seed 100 chooses thresholds; seeds 101–109 confirm them. After 1,024 steps at $\beta = 0.01$, three 2,048-step cycles linearly raise β from zero to one in their first halves, hold it at one in their second halves, and reset. Total length is 7,168 updates. Training is independent of monitor requests. References every 64 steps define loss when both normalized quantities are at most 0.5, and recovery when both are at least 0.65, each on two consecutive checks. The event time is the second check. There are 41 events with a complete 512-step detection horizon and two late events scored separately.

The monitor requests a reference when the absolute log change of its feature from the previous request exceeds its pilot-selected threshold, respecting a 128-step refractory period and a budget of 6, 12, or 24 extra checks. Every method also receives one initial check. For each budget, thresholds are selected on the pilot from 0.025, 0.05, 0.075, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5, 0.7, 1, 1.5, 2, and 3. Features are evaluated every 64 updates in this cyclic experiment; the earlier preservation experiment uses stride 128. Decisions use no future values. Periodic checks use evenly spaced grid points with no favorable phase selection.

At budget 12, MG makes 9.44 requests on average. Matching a periodic schedule to the number of requests for each seed yields 34 detections, compared with MG’s 28; this post-hoc comparison uses request totals but no event times. Against the main periodic comparator, MG’s mean per-seed recall is lower by 24.4 percentage points, with bootstrap interval $[-36.7, -10.0]$. At budget six, MG detects 18 events versus 12 for periodic checks, although a rule based on the known training schedule also detects 18. MG is therefore competitive at the smaller budget but does not improve monitoring overall in this test.

Table 13: Cyclic VAE: detected events out of 41 at all tested budgets. Primary budget is 12.

Method	$B = 6$	$B = 12$	$B = 24$
MG	18	28	36
Standard deviation	13	20	38
Spectral entropy	17	28	35
Training KL	8	13	31
Schedule	18	18	26
Periodic	12	38	40

E ADDITIONAL LEARNING SETTINGS

MNIST. A 15,018-parameter tanh MLP is trained on a fixed balanced 2,040-image subset after downsampling to 14×14 . SGD uses momentum 0.9, batch 64, and no weight decay. At step 2,048 of 4,096, one paired branch reduces the learning rate from 0.1 to 0.01. A fixed 100-image loss probe is selected on the pilot. Detrended covariance rank declines more than the reference on all five new seeds; MG does so on four of five, with median contrast only 4.8%. The larger-window sensitivity is more favorable but does not replace the primary result.

Grokking. The four positive sketch runs comprise modular addition $y = (a + b) \bmod 113$ on seeds 42–44 and permutation composition in S_5 on seed 42. Training fractions are 0.30 and 0.50, respectively; zero-decay reference runs use the corresponding seed-42 tasks. A one-layer transformer has width 128, four 32-dimensional heads, MLP width 512, no layer normalization, and float64 arithmetic. AdamW uses learning rate 10^{-3} , betas (0.9, 0.98), batch 256, and decay 1.0 (addition) or 0.2 (S_5). Training lasts 20,000 or 15,000 loop iterations. The historical S_5 implementation performs two optimizer updates per loop iteration; this difference is retained when reporting its times. The unregularized full-batch comparison uses a width-500 quadratic perceptron at modulus 97, a 50% training split, and plain gradient descent on mean squared error. Random projections of parameter and logit trajectories support detrended covariance analysis. A sharp collapse and later recovery aligns with generalization in four positive regularized runs, with two references; a memorizing zero-weight-decay run can show simplification elsewhere. The short-window parameter-norm analysis uses 60 logged samples at the centers of the trajectory windows. Its grid crosses $E \in \{4, 6, 10\}$, $\tau \in \{1, 2, 4\}$, $k \in \{5, 20\}$ and embedding/ACF exclusion. The headline cell maximizes E subject to $(E - 1)\tau \leq 60/4$, retaining the frozen k and exclusion rule, and breaking ties by smaller τ : $E = 10$, $\tau = 1$, $k = 20$, embedding-span exclusion. This rule does not select the largest observed effect. The scalar decline is a change statistic at that resolution. Its 24/26 usable cells vary analysis settings on the same runs. The signal is not a unique indicator of generalization, and scalar roughness is an alternative explanation in this setting. We retain this as a complementary example rather than the sole applied validation. Linear detrending removes the scalar separation in the matched-window analysis. Covariance PR can change with drift, correlation time, or coherent motion even without a change in active dimension; random-walk and Ornstein–Uhlenbeck trajectory PCA illustrate this distinction (Antognini and Sohl-Dickstein, 2018). We have not established that the grokking effects persist against nulls matched jointly in temporal correlations and nonstationary drift. The observations therefore identify a change in trajectory and scalar statistics, not its unique dimensional cause.

Walker2d. PPO policies reference six actuators in Walker2d-v5. Fixed-policy evaluations use a 512-step warm-up and 4,096 recorded steps. In phase-conditioned variants, the observation is augmented by sine/cosine phase and the reward is changed to encourage an intended periodic behavior. Subsequent comparisons use a common initial locomotion checkpoint with separate fine-tuning seeds, not independently trained starting skills. Scalar joint and action logs are analyzed alongside full-state return errors, inter-cycle variation, and responses to injected perturbations.

The smoothing objective is

$$L = L_{\text{PPO}} + \frac{\lambda}{6} \mathbb{E} \|\bar{a}_\theta(s_{t+1}) - \bar{a}_\theta(s_t)\|_2^2,$$

where $\bar{a}_\theta(s)$ is the policy mean clipped to $[-1, 1]$ per actuator. Episode boundaries are excluded. The environment reward and executed commands receive no extra smoothing. Separate tracking variants change the reward to $r'_t = r_t - \lambda(1 - \exp[-e_t^2/(2\sigma^2)])$, where e_t measures distance to the recorded reference cycle, either at its nearest point or at the prescribed phase. The reference has 152 samples; the external phase period is approximately

152,215 reference steps. The initial $\sigma^2 = 0.116779$ saturated; a separate held-out development gave 38.477 for the broader tracking experiment.

Actor and critic use separate 64–64 tanh networks, eight environments, 512-step rollouts per environment, batches of 256, five PPO epochs, learning rate decreasing from $3 \cdot 10^{-5}$ to $3 \cdot 10^{-6}$, clip 0.1, target KL 0.01, discount 0.99, GAE 0.95, entropy coefficient zero, value coefficient 0.5, and gradient clipping 0.5. Normalizations are frozen. Fine-tuning branches use a common initial checkpoint after 655,360 transitions, then 1,048,576 additional transitions per branch; their seeds do not represent independently learned initial policies.

Moderate action-smoothing penalties give the most reproducible scalar result: two five-seed series show lower aggregate action-log MG. In the final five-seed comparison, penalty strengths 0.25 and 1 give median MG/reference ratios 0.801 and 0.604, with a decline on every seed. Normalized increments also decline in four and five seeds and are cheaper. In the stricter full-gait experiment, only two of five pairs meet the joint improvement criterion, and only one pair has lower MG under every prescribed reconstruction. Thus a simpler command signal is not sufficient evidence of a simpler or more stable whole-body gait. Timing advantages below concern repeated perturbation probes, not these elementary scalar competitors.

E.1 Selecting policies for actuator shifts

We evaluated historical PPO policies on new nominal and deployment episodes. The 16 candidates per fine-tuning seed cross smoothing strengths 0, 0.25, 1, 4 with checkpoints at 262,144, 524,288, 786,432 and 1,048,576 transitions. Development seed 231 is excluded from the summary; seeds 232–235 provide the four confirmation units. All policies share one initial learned skill. Earlier results informed the setting, so this is new-outcome evaluation on existing policies, not an independent sample of newly learned skills.

Each selector receives three nominal episodes (reset seeds 83001–83003), with 256 warm-up and 2,048 measured reference steps per episode. Eligible candidates complete at least two episodes and have mean zero-padded return at least 90% of the largest nominal mean. MG selects the lowest median estimate from the scalar action norm over complete episodes, using $E = 20$, $\tau = 8$, $k = 20$, $T = 312$, and dither seed 123. Entropy, normalized squared increments, and minimum scalar return error over lags 20–250 use the same log. Action smoothness J_1 uses all six commands. Reward selection maximizes the mean over all episodes, including falls. Every method has the same eligibility gate. The fixed pilot choice is smoothing strength 0.25 at the final checkpoint, falling back to nominal reward if ineligible.

Deployment applies $v_t = (1 - \alpha)u_t + \alpha v_{t-1} + \epsilon_t$, with $v_{-1} = 0$, commanded action u_t , and independent Gaussian components of ϵ_t with standard deviation σ . The environment receives v_t clipped to $[-1, 1]$; the recursion retains the unclipped value. The six conditions cross $\alpha \in \{0.05, 0.1, 0.2\}$ and $\sigma \in \{0, 0.02\}$ and apply also during warm-up. Five new reset seeds 84001–84005 per condition give 30 target episodes per candidate. Return is the measured reward sum divided by 2,048, with a zero remainder after falls. Selectors never use confirmation target outcomes; the oracle does. The sampling interval is 0.008 seconds.

Table 14: Policy selection under hidden actuator shifts: mean over four fine-tuning seeds. Completion is over all 120 selected target episodes. Random denotes expected uniform choice among eligible policies; the oracle uses target returns.

Selector	Return	Completion
MG	4.313	90.8%
Nominal reward	4.005	80.0%
Spectral entropy	4.480	97.5%
Scalar recurrence	4.398	95.0%
Action smoothness	2.999	47.5%
Norm. increments	3.064	49.2%
Fixed pilot	4.310	90.8%
Random eligible	4.286	90.9%
Oracle (eligible)	4.671	100.0%

MG exceeds nominal-reward selection on three of four seeds, with paired mean difference 0.308 (7.7%); a two-sided sign-flip test gives $p = 0.375$. It does not outperform spectral entropy, and its mean advantage over the fixed pilot is only 0.0025. Reset repetitions and candidate checkpoints are correlated and are not independent statistical units.

MG-based selection therefore provides a descriptive benefit over one common rule, not established superiority over competing selectors.

On an Intel i5-12500H CPU with one numerical thread, a 2,048-point MG calculation takes 0.516 seconds, compared with 0.00014 for entropy and 0.0062 for scalar recurrence. Three nominal episodes take 5.52 seconds, while 30 target episodes take 44.7 seconds on the benchmark policy. Nominal acquisition plus three MG calculations is approximately 7.06 seconds; these component medians do not establish a universal speed advantage, and entropy uses the same acquisition budget. Training all candidate policies is a shared sunk cost. Choosing an earlier stored checkpoint does not demonstrate causal early stopping or curriculum improvement.

An earlier stress test selected among four final policies using delay-free observations and applied delays of 1–3 entire reference steps after warm-up. On five seeds, MG and the pilot-selected fixed strength 1 choose the same policies, with return 0.541 versus 0.319 for nominal reward, but only 2/150 target episodes finish without falling. This severe-shift test is retained as a negative deployment result.

F TIMING PROCEDURES AND INFORMATION REQUIREMENTS

Timing separates simulation/training, recording an observable, analysis of an existing record, and extra diagnostic interventions. The comparisons are local implementation benchmarks with the listed configurations. They do not establish a universal complexity ordering among all algorithms for the same scientific question.

F.1 Computing infrastructure and provenance

The FORCE timing report identifies an Intel Core i5-12500H CPU. Its saved environment specifies Windows 10 (build 18363), Python 3.13.12, NumPy 2.4.3, SciPy 1.17.1, and one BLAS thread; the benchmark uses float64 arithmetic and three warmed sequential repetitions. No GPU is used in this benchmark.

The cyclic-VAE benchmark records CPU execution on Windows 10 (build 18363), Python 3.13.12, and PyTorch 2.10.0+cpu, with two PyTorch threads, one estimator thread, and forty interleaved repetitions. Its metadata do not identify the CPU model. The Walker timing metadata record the same operating system and Python version, an Intel64 Family 6 Model 154 Stepping 3 processor, one thread, and five repetitions.

For the new ten-seed CNN confirmation, the local machine identifies an Intel Core i5-12500H (12 physical cores, 16 logical processors); the training script uses CPU execution with six PyTorch threads. This machine identification was collected during manuscript revision on October 2, 2026 and does not retrospectively establish the hardware of every earlier experiment. The historical CNN timing output does not record a CPU model. Absolute runtimes and speed ratios therefore describe the documented local implementations; a complete hardware inventory for all historical runs is unavailable.

F.2 Benchmark-specific timing and storage

CNN. The saved graded result reports median MG processing of 9.49 ms per 1,000-point window. A float32 scalar record of 10,000 updates takes 39.06 KiB; the corresponding 14,666-dimensional trajectory takes 559.46 MiB. Parameter-norm acquisition is an $O(P)$ reduction, not zero-cost logging. CountSketch update references in ResNet use 1,024 stored coordinates per update and require computing the sketch. A streaming reference would change the storage comparison.

FORCE. The 256-unit endpoint benchmark replays fixed-checkpoint autonomous trajectories. Median MG times before/after training are 0.835/0.598 s; full-spectrum times are 12.006/11.551 s. With both $E = 20$ and $E = 40$ calculations, scalar analysis takes 2.233/1.213 s. Scalar recording costs 0.102/0.166 s, including rollout; this is separate from analysis. The largest exponent costs 0.198/0.153 s and elementary scalar baselines 0.0009/0.0015 s. The full spectrum supplies information that a single exponent or scalar score does not, and MG does not recover the exponents themselves.

In the 1,000-unit generator, analysis of four 8,192-point windows takes 8–23 s across learned-task medians. The twelve-vector Lyapunov calculation takes 41–66 s; full-state PCA is about 0.2 s. The advantage over PCA is its input requirement, not time. The difference from the CNN’s millisecond windows is material and follows from record/configuration and implementation costs.

Coupled oscillators. A separate benchmark evaluates dense phase networks of 32,64,96,128 oscillators at two coupling levels using a shared saved 1,536-point trajectory for each comparison. Scalar analysis computes MG and LB together; the full spectrum integrates all tangent directions with QR steps. Medians use five scalar and three spectrum repetitions. At size 128, scalar times are 0.061/0.049s and full-spectrum times 1.309/1.124s. Scalar storage is 128 times smaller. At size 32, the full spectrum is faster. An additional same-seed 64-unit comparison with 4,096-point scalar windows also favors the spectrum (roughly 1.0–1.4s versus 1.6–2.2s), so the short-window speedup is not universal. The same-seed coupling sweep shows a collective contraction of the spectrum and MG approaching one after synchronization; finite-time exponent thresholds are not exact dimension labels. This is a dynamical-system benchmark, not neural-network training.

VAE. The cyclic benchmark uses one warmed process, two PyTorch threads, one estimator thread, and forty interleaved repetitions. Median costs are 7.134ms per fixed probe, 12.890ms per MG window, and 209.323ms per combined reference. Cost totals multiply component medians by the actual numbers of operations, including 7,168 probe acquisitions and 105 feature calculations. They are estimates of deployment cost, not separately timed complete monitoring runs. Other jobs may share the machine. At budget 12, acquisition alone costs 51.14s, leading to the 54.68s MG total. Cheap statistics using the same probe incur the same acquisition cost. Periodic checks require no such probe stream.

Walker2d. In two fixed-horizon comparisons, MG of an existing scalar is approximately 21–25 times cheaper than repeated perturbed simulations; including the doubled-embedding check reduces this to about 10–11 times. These probes answer questions about stability that MG alone cannot answer. On the final scalar benchmark, MG costs 369ms per window, versus 0.103ms for spectral entropy, 3.782ms for a scalar return statistic, and 0.048ms for normalized increments. The data-saving property is shared by all four scalar methods.

G HOW THE EXPERIMENTS SUPPORT THE METHOD

The synthetic tests connect MG to known phase structure and identify its finite-resolution limits. The applied tests ask a separate question: does the same estimator detect a change confirmed by an independent measurement? Its value for that task depends on reliable detection, false positives, and cost, rather than exact integer recovery. We report comparisons where MG adds information or saves computation, alongside cases where a simpler statistic is sufficient or better. The resulting evidence supports the specified observables and tasks; a new application requires the same kind of independent check.

H FORECAST SELECTION AND SAMPLED-QUERY ESTIMATION

Training and observable. This experiment uses new 256-unit networks with the FORCE/RLS dynamics described in Appendix C. The seven task families are T1–T4, H2, H4, and M4. Each network trains for 40,000 updates with recurrent gain 1.2, integration step 0.1 and RLS updates every two steps; RLS starts at the identity. Target frequencies and random phases follow the generator construction above. An eighth, untrained reference has gain 1.5; it is not assumed to be chaotic and can settle to a fixed point. At frozen weights, each run has 5,000 burn-in steps and 8,192 recorded samples. The scalar is $\tanh(h_{t,2})$, using zero-based neuron index two. We use observations 4,096–6,143 as the prefix and 6,144–6,175 as the future target. No future values enter normalization, feature extraction, candidate fitting, or selection.

Training fidelity is not an inclusion criterion. In particular, the smaller networks do not reliably learn the higher-phase targets. Forecast errors concern their realized activity, not proof of the intended dimension. This experiment therefore evaluates a downstream decision on a specified family of trained networks rather than a new theorem about phase recovery or an application to arbitrary language-model logs.

Eight candidate forecasts. The pool comprises the last value, prefix mean, repetition at lag 314, ridge regression with 8 or 48 lags, five-nearest-neighbor regression with 8 or 48 lags, and ridge regression on 64 random Fourier features of a 48-lag input. Forecasts are direct 32-output regressions, fitted on all complete input/target pairs inside the prefix. Ridge uses penalty one and intercept penalty 10^{-8} on standardized inputs. The Fourier feature map has bandwidth $1/48$ and random seed 42. Nearest neighbors have uniform weights. Learned predictions are clipped at ± 10 prefix standard deviations. NMSE is squared forecast error averaged over

the 32 future values and divided by prefix variance; no square root is taken. Prefix standard deviation below 10^{-10} triggers a shared persistence fallback, scored with variance floor 10^{-12} for every selector.

Selector training and search history. Each selector predicts the eight candidate losses and picks their minimum. Training uses seeds 601–618 (144 task records); three-fold cross-validation groups by seed. Four candidate selector architectures are considered equally for every feature set: decision trees of depth 2/3 with minimum leaf size eight, and 100-tree random forests of depth 3/6 with minimum leaf size five. Random seed is 42. Missing-value imputation and indicators are fit inside the training folds. The MG selector is a depth-two tree trained on MG alone and generally selects nearest-neighbor prediction for cyclic records and ridge regression for more complex records. The task label and target phase dimension are never selector inputs.

The primary observable/horizon was chosen during an exploratory search, not specified before all experiments. Earlier screens used historical generators, multiple horizons and neurons, real sensor forecasting, and other downstream tasks. Seeds 619–630 identified the short-horizon candidate; seeds 631–650 tested it, after which stronger geometric competitors were added. All full-MG and geometric selector rules were then frozen before seeds 651–680 (30 new groups, 240 records). The sampled-query variant was proposed after that study’s cost measurement and tested with the same frozen selectors on seeds 681–700 (20 further groups, 160 records). The two evaluation cohorts are reported separately. A seed shares initialization across task families; statistical resampling groups the eight tasks of each seed. The bundled procedures retain the exploratory decisions and unsuccessful cases; the results are conditional on this network family and task mixture.

Competitors and information access. All feature-based competitors use the same development runs and selector search. Cheap features are spectral entropy, normalized squared increments, lag-one correlation, minimum scalar recurrence error over lags 6–288, spectral peak concentration, linear trend, standard deviation, and mean. The geometric audit includes TwoNN and original Levina–Bickel pooling on the same delay cloud, its covariance participation ratio, permutation entropy of order five, and sample entropy with template length two and tolerance 0.2 standard deviations in Chebyshev distance. Held-out model search fits all eight candidates on four rolling prefixes with 32-point validation targets covering the last 128 known observations, then refits the chosen candidate on the entire prefix. Learned selectors receiving these validation errors have additional input and fitting cost. “All except MG” combines the cheap features, validation errors, and non-MG geometric features; “all including MG” adds MG to that same set. The oracle uses future errors and is only a lower bound. Table 15 includes these stronger alternatives.

Table 15: Mean NMSE for all forecast selectors. Columns use distinct seed cohorts; lower is better. Earlier MG was fit during the initial generator screen. The second cohort supports the query-budget comparison.

Selector	30 seeds	20 new seeds
MG	0.03138	0.03637
Earlier MG rule	0.03219	—
Calibration-best fixed	0.04475	0.06041
Held-out model search	0.03224	0.04074
Cheap features	0.04517	0.05923
Cheap + held-out search	0.03617	0.03379
Held-out errors	0.03997	0.03424
Held-out errors + MG	0.03423	0.03167
Cheap + MG	0.03138	0.03637
Cheap + held-out search + MG	0.03534	0.03994
TwoNN	0.03540	0.04364
Levina–Bickel	0.04054	0.04732
Delay covariance PR	0.04397	0.06098
Permutation entropy	0.04484	0.05384
Sample entropy	0.04718	0.05954
All features except MG	0.03669	0.03939
All features including MG	0.03420	0.04071
Oracle (future targets)	0.00759	0.00847

On the first cohort, MG gives NMSE 0.03138 versus 0.03224 for ordinary held-out search; seed-bootstrap uncertainty includes no difference. On the second cohort, MG gives 0.03637 versus 0.04074: a 10.7% lower sample mean, but the paired improvement interval is $[-0.00729, 0.01758]$ over 20 seed groups (20,000 bootstrap resamples,

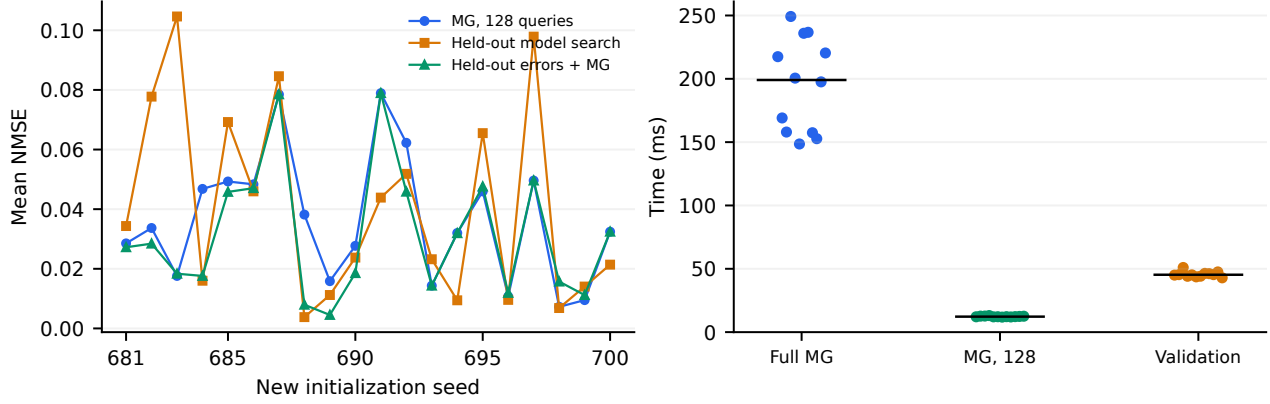


Figure 4: Forecast selection: quality and cost. Left: each new seed group averaged over eight tasks; lower NMSE is better. Right: medians over seven timing repetitions for each of 12 matched inputs; black bars are medians over inputs.

random seed 811). MG wins on nine of those 20 groups. Cheap features plus learned held-out-error selection achieve 0.03379 and held-out-error features plus MG achieve 0.03167. These results support usefulness without establishing statistical dominance over validation or other learned selectors. The first-cohort trained-arms-only analysis also retains the advantage over TwoNN (0.03542 versus 0.03943), but it does not remove the uncertainty against the strongest methods.

Sampled-query MG. Use $E = 20$, $k = 20$, the adaptive lag of Appendix B, an autocorrelation Theiler rule capped at 150, and dither seed 123. Standardization, floors, and temporal exclusion match the original kernel. For n delay vectors, select $q = \min(128, n)$ distinct query indices uniformly across the record, with zero-based indices $\lfloor j(n-1)/(q-1) \rfloor$. All n vectors remain candidate neighbors. With original time indices retained in the exclusion, compute

$$\hat{d}_{\text{MG}}^{(q)} = \frac{q(k-1) - 1}{\sum_{i \in Q} \sum_{j=1}^{k-1} \log(r_{ik}/r_{ij})}. \quad (1)$$

This is a computational approximation, not a new finite-sample guarantee. It agrees with the MG formula when Q contains every query. The same numerical-floor flag is evaluated over queried distances/sums. Insufficient neighbors, nonfinite values, and constant records remain unavailable measurements. After imputation, the unchanged full-MG selector maps the approximation to a candidate forecast.

The implementation computes squared Euclidean distances in blocks of 64 queries, applies the Theiler exclusion, partitions the nearest k admissible distances, and pools their log ratios. Distance work is $O(qnE)$ with bounded query-block memory. The speed comparison is against the original KDTree implementation, so it reflects both query reduction and implementation; it does not establish an optimal nearest-neighbor algorithm or a universal complexity improvement. Both implementations use the same full neighbor cloud.

On all 160 records of the new cohort, $q = 128$ retains exactly the full-MG model choice and therefore the same forecast error. It does not produce an identical numerical estimate in general. The sensitivity budgets $q = 64$ and 256 also preserve those choices on this cohort; 128 is the primary budget, not a test-selected optimum. Twelve audited records check exact full-query agreement, tree/block neighbor-distance agreement, and affine rescaling; constant and nonfinite inputs are rejected. Forecast-choice agreement alone does not certify absolute active dimension.

Paired runtime procedure. Serial measurements use an Intel Core i5-12500H CPU, one numerical thread, seeds 681, 690, and 700 crossed with tasks T1, T2, H4, and T4, seven interleaved repetitions per input after warm-up. Medians per input are formed before aggregating the 12 inputs. Overall median times are 199.1 ms for full MG selection, 12.2 ms for $q = 128$, and 45.3 ms for held-out model search. Median paired speed ratios are 16.3 and 3.7, respectively. Each includes the feature, selector, and final forecast fit. The held-out search performs

32 validation fits and one final fit; MG fits only its selected candidate. Disk access, acquisition of the common prefix, generation of the development bank, and offline selector training are excluded. Thus the benefit concerns repeated decisions with an already available record and a previously trained selector; direct cheap statistics can still be faster.

I TRAINING INTERVENTIONS: BENEFITS AND BASELINE COMPARISONS

We also tested whether MG can choose when to stop, freeze, or compress a model during training. These tests address practical utility but did not establish MG-specific superiority. They are retained to delimit the successful forecast-selection and RL results.

MNIST images are average-pooled to 14×14 . A fixed balanced split uses 2,000 training images, 500 disjoint clean held-out evaluation images from the original training pool, and 2,000 official test images (split seed 1006). The network is a 196–128–128–10 ReLU MLP trained by Adam at 10^{-3} , batch 64, for 2,048 updates. For stopping experiments, exactly 40% or 60% of training labels are replaced by a randomly selected wrong class. Each initialization seed has its own corruption mask. Observers are parameter norm and loss on a fixed random subset of 128 training examples with their possibly corrupted labels, not clean probe labels. The initial single-class probe pilot is superseded by this corrected protocol.

Candidates act at updates 256, 512, 768, 1,024, or 1,536; no threshold crossing means continuation to 2,048. Trailing windows have 256 samples, $E = 10$, $\tau = 1$, $k = 10$, and $T = 19$. MG competes with mean level, normalized trend, detrended standard deviation, increments, entropy, and lag-one correlation. All select one of the two observers and threshold direction/level with the same pilot search. Thresholds maximize pilot clean-validation accuracy minus 0.02 times the arithmetic budget. Seeds 0–2 choose the rules; seeds 10–17 evaluate them. The full base confirmation paths were generated before the rule file was saved; rule fitting reads only pilot records, and this ordering is disclosed rather than described as prospective preregistration. A separate clean-validation checkpoint choice is an offline comparator, not a causal oracle for test accuracy. The corrected fallback uses the final metric row, not a nonexistent feature row.

On clean training, freezing disables gradients through both hidden layers and continues fitting the final head. Pruning physically reduces both hidden layers to width 64, retaining units according to incoming-row norm times outgoing-column norm, copying retained weights/biases and sliced Adam moments. Each selected action time is executed from its saved checkpoint with the same minibatch RNG, then trained to update 2,048. Post-action outcomes are actual continuations, not inferred from the unchanged base path.

At 40% noisy labels, MG stopping yields 84.75% test accuracy at 448 updates on average, versus 64.11% at the full budget; the mean-level rule yields 84.90% at 384 updates. At 60%, MG and fixed early stopping choose update 256 and both yield 76.58%, versus 43.83% for the full budget. For clean-data pruning, MG yields 92.44% accuracy at 57.2% of the dense arithmetic budget, versus 92.29% for uninterrupted dense training. Fixed-time pruning yields 92.52% at 62.8%, while entropy gives 92.38% at 55.3%. These comparisons demonstrate useful interventions, but not that MG is their uniquely informative trigger. Development uses clean held-out evaluation on pilot seeds and noise-specific rules; only application to later runs avoids clean held-out evaluation. Scalar acquisition and monitoring remain extra costs.

REFERENCES

- Joseph M. Antognini and Jascha Sohl-Dickstein. PCA of high dimensional random walks with comparison to neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- Christoph Bandt and Bernd Pompe. Permutation entropy: a natural complexity measure for time series. *Physical Review Letters*, 88(17):174102, 2002. doi: 10.1103/PhysRevLett.88.174102.
- E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954. doi: 10.1093/biomet/41.1-2.100.
- Joshua S. Richman and J. R. Moorman. Physiological time-series analysis using approximate entropy and sample entropy. *American Journal of Physiology–Heart and Circulatory Physiology*, 278(6):H2039–H2049, 2000. doi: 10.1152/ajpheart.2000.278.6.H2039.

Table 16: Eight-seed intervention results, including strong simple comparisons. MAC ratio is an approximate matrix-arithmetic budget relative to uninterrupted dense training; it excludes logging, monitoring, nonlinearities, optimizer work, and memory overhead and is not a wall-time speedup.

Task	Rule	Accuracy (%)	MAC ratio
Stop, 40% noise	MG	84.75	0.219
Stop, 40% noise	fixed	84.72	0.250
Stop, 40% noise	level	84.90	0.188
Stop, 40% noise	entropy	84.67	0.203
Stop, 40% noise	never	64.11	1.000
Stop, 60% noise	MG	76.58	0.125
Stop, 60% noise	fixed	76.58	0.125
Stop, 60% noise	level	76.58	0.125
Stop, 60% noise	entropy	76.58	0.125
Stop, 60% noise	never	43.83	1.000
Freeze, clean	MG	91.53	0.535
Freeze, clean	fixed	91.32	0.515
Freeze, clean	level	91.32	0.515
Freeze, clean	entropy	91.32	0.515
Freeze, clean	never	92.29	1.000
Prune, clean	MG	92.44	0.572
Prune, clean	fixed	92.52	0.628
Prune, clean	level	92.50	0.618
Prune, clean	entropy	92.38	0.553
Prune, clean	never	92.29	1.000

Jaroslav Stark. Delay embeddings for forced systems. I. Deterministic forcing. *Journal of Nonlinear Science*, 9(3): 255–332, 1999. doi: 10.1007/s003329900072.

Jaroslav Stark, David S Broomhead, ME Davies, and J Huke. Delay embeddings for forced systems. II. Stochastic forcing. *Journal of Nonlinear Science*, 13(6):519–577, 2003. doi: 10.1007/s00332-003-0534-4.